Cross-Platform Input System
The Sample Project Mini-Cookbook

This free booklet introduces the newly implemented `InputSystem` for Knight, a thin framework built on top of raylib and used by all the sample projects for the book. The input system was added after the book was published, so this booklet is a companion resource for readers who want to explore the new feature.

In this mini-booklet, you will learn how to use Knight's InputSystem and define input actions for your gameplay controls.

Before this input system was added, the sample projects used raylib's mouse and keyboard functions directly. Those functions work well on desktop PC, and they are also available when you use raylib in your own macOS or Linux project. So why add another layer of code for game input?

Well, real games often need more flexibility in how input controls are assigned and interpreted:

Let players customize their controls

Many desktop games let players choose the keys they prefer in a settings menu:

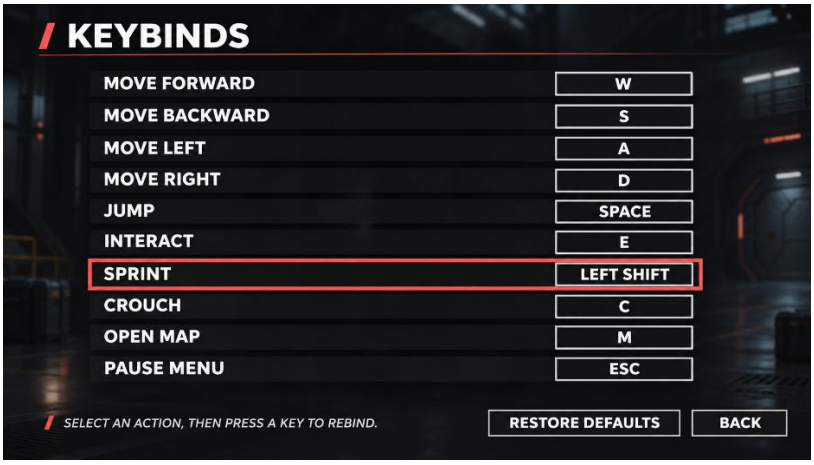


Figure 1, Many games allow player to customize the actual hotkey for control

A player might bind an action to a different key or use a key combination, such as pressing X while holding the left Control key. This flexibility is especially useful in games with many input commands, such as MMORPGs and real-time strategy games.

Support the same gameplay control across devices

A game may need to support keyboard and mouse on a computer and on-screen touch controls on a phone or tablet. Raylib runs across these platforms, while Knight's new input layer gives gameplay code a shared way to read the keyboard, mouse, and touch controls described in this booklet. A game that adds gamepad input can connect it to the same actions through an appropriate input binding.



Figure 2 – Running same sample project on Windows and Android phone

Combine several inputs into one gameplay action

Some controls depend on a sequence or gesture rather than one button press. For example, a special attack might trigger when the player presses X three times within three seconds. Handling every key, timing rule, and gesture directly inside Update() can make that function large and difficult to maintain. A separate input layer provides a place to organize these controls and keeps gameplay code focused on the resulting action.

For a small sample with simple input needs, raylib's input functions remain a good choice. If the only interaction is a mouse click, for example, there may be no need for another abstraction; on a touch device, a simple tap can be handled as a mouse-style click.

When a real game development task needs the same gameplay control to work with keyboard input and on-screen controls, or has many actions and combinations to

manage, Knight's `InputSystem` provides a useful starting point. You can use it as part of Knight or adapt its code for another raylib project.

Before we begin

You will need the knowledge from *Chapter 1*, especially how a Knight application starts and updates. Familiarity with raylib keyboard, mouse, and touch input is helpful, but the booklet explains the input concepts used in its examples.

The cookbook focuses on the current Knight input API. The input layer is built on raylib and separates device reading from gameplay control definitions. It does not require you to replace the rest of the Knight framework.

Getting the project up and running

First, pull the latest updates from the book's GitHub repository: https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main. Open the Visual Studio solution in Visual Studio Community 2022, rebuild it, and run a BonusGameWorld02 sample.

Start with the desktop keyboard-and-mouse profile. Then press F4 to turn on touch emulation and inspect the sample's virtual controls.

Press F4 again to return to keyboard and mouse. This lets you compare both control schemes without changing the gameplay code or connecting an Android device.

Why use an input system

A direct call such as IsKeyDown(KEY_W) works well when a sample has one control scheme and only needs to run on desktop. As a game grows, the same physical input may need to trigger different gameplay actions, and one gameplay action may need to work from several devices.

For example, a player may move with WASD on a computer and with a virtual pad on a phone. In a game that supports gamepad input, an attack action might also be bound to

a gamepad button. If each gameplay function checks every device separately, the Update() code becomes tied to particular devices and harder to customize.

Knight's `InputSystem` provides a common layer for these cases. It reads the active device through an input backend, updates named actions and axes once per frame, and exposes the resulting state to gameplay code. It also keeps many older key-based samples working through Knight's input wrappers.

Understanding InputSystem and InputAction

`InputSystem` is the object that gathers and interprets input. Each Knight application owns an `InputSystem` through its `Input` member. Code inside a Knight-derived class can use `Input` directly; components, cameras, and other code outside that class can access the same system through `KnightInput()`.

The low-level reads are isolated behind `InputBackend`. In the sample application, `RaylibInputBackend` forwards keyboard, mouse, touch, screen, and focus information from raylib. This separation lets the `InputSystem` process input consistently and lets tests supply a scripted backend instead of opening a game window.

An `InputAction` represents a button-like intention in the game, such as **Jump** or **Attack**. It is not tied to a single key. Bind one or more keys to the action, then ask whether the action went down this frame, is currently held, or was released.

```
InputAction jump = Input.RegisterAction("Jump");

Input.BindKey(jump, KEY_SPACE);

Input.BindKey(jump, KEY_W);


if (Input.Pressed(jump)) { /* start the jump */ }

if (Input.Down(jump))    { /* continue while held */ }

if (Input.Released(jump)){ /* finish the jump */ }
```

`RegisterAction()` returns the existing action if the same name has already been registered. `Pressed()` and `Released()` report the start and end of an input. They also account for the key press and release events, so a brief tap between frames is not silently lost.

An `InputAxis2D` represents direction, commonly used for player movement or camera control. Its x and y values follow screen convention: positive x points right and positive y points down. Keyboard directions produce -1, 0, or 1; a touch pad can supply an analog value.

```
InputAxis2D move = Input.RegisterAxis2D("Move", KEY_W, KEY_S, KEY_A,
KEY_D);

Input.BindAxisKeys(move, KEY_UP, KEY_DOWN, KEY_LEFT, KEY_RIGHT);

Input.BindAxisTouchPad(move, TouchPad::Left);


Vector2 direction = Input.Axis(move);
```

The built-in `MoveAxis()` already includes WASD and the arrow keys. `SecondaryAxis()` uses I, K, J, and M. These defaults do not create a touch pad automatically; a sample binds a pad only when it needs one.

Configuring input in a sample

A Knight sample configures its controls by overriding `OnConfigureInput()`. Knight calls this method once during `Start()`, after creating the window and before creating default resources. Register actions and axes there, and add only the virtual controls the sample needs. A sample that adds no touch controls displays none.

In the BonusGameWorld02 sample pattern, the setup assigns the movement axis, binds a left virtual pad, and registers an attack action that works from both the keyboard and a touch button. Gameplay then reads the named control state without checking the device.

```
GameInput.Move = Input.MoveAxis();

Input.BindAxisTouchPad(GameInput.Move, TouchPad::Left);


GameInput.Attack = Input.RegisterAction("Attack");

Input.BindKey(GameInput.Attack, KEY_SPACE);

Input.AddTouchButton(GameInput.Attack, "ATTACK",
TouchButtonAnchor::TopRight);
```

```
// In gameplay code, on every platform:
const Vector2 move = KnightInput().Axis(GameInput.Move);
if (KnightInput().Pressed(GameInput.Attack)) { /* attack */ }
```

Keep each virtual button close to the action it triggers in the setup. Use short labels that fit inside the button, and place buttons in the top corners when a movement pad occupies a bottom corner.

For a sample that already reads raylib-style keys, `AddKeyPad()` and `AddKeyButton()` can add touch controls without rewriting those checks. This is useful for gradually adopting the `InputSystem` while preserving familiar input code.

```
Input.AddKeyPad(TouchPad::Left, "Walk", KEY_W, KEY_S, KEY_A, KEY_D);
Input.AddKeyButton(KEY_SPACE, "ATTACK", TouchButtonAnchor::BottomRight);
```

Adding on-screen controls

On-screen controls are shown and read only when the active profile is `Touch`. Knight provides two pads and corner-anchored buttons. A pad that has no axis or key mapping is hidden and ignores touches.

Movement and camera pads

Bind a pad to an axis when the game reads a named direction. The pad returns an analog value within a unit circle. A small dead zone around the center prevents accidental drift. A held keyboard direction takes priority over the pad on the same axis.



Figure 3 – Explaining the dead zone

A fixed pad requires the player to touch its displayed area. A floating pad appears where the player first touches in the lower part of the screen and follows the thumb as it moves beyond the pad edge. Use a floating pad when players benefit from being able to place their thumb wherever it is comfortable.

A key pad instead translates its eight directional sectors into key states. A diagonal presses two directions. This makes it a convenient bridge for existing code that already checks keyboard keys.

Action buttons

`AddTouchButton()` binds a visible button to a registered action. `AddKeyButton()` binds it to a key and creates an action for that key. Buttons are anchored to one of the four corners and automatically occupy the lowest available slot. You can supply a slot yourself when a layout needs more control.

A finger is assigned to a control when it first touches. Sliding off a button does not release it, and sliding onto a second control does not transfer the finger. A finger on a control is kept separate from fingers used to drag the view or perform a pinch gesture.

Control sizes are measured in density-independent pixels, so the controls remain a similar physical size across screens with different pixel densities. Their layout is recalculated as the display changes. The default control scale is 1.2; set another scale from the sample when the controls need to be larger or smaller.

```
Input.SetControlScale(1.0f);  // smaller controls

Input.SetControlScale(2.0f);  // very larger controls
```

Use `SetVirtualControlsVisible(false)` to hide controls without removing their bindings. Use `ClearVirtualControls()` to remove all buttons and detach both pads. Keyboard bindings remain in place.

Pointers and gestures

Some controls are naturally expressed as pointer movement rather than a named button. `PrimaryPointer()` represents the left mouse button on the keyboard-and-mouse profile and the first free finger on the touch profile. Fingers already assigned to on-screen controls are not treated as pointers.

`SecondaryPointer()` represents the right mouse button on desktop. In the touch profile it follows the free finger used for a one-finger drag, which allows camera orbit. A long press of about half a second without moving more than 12 dp triggers its pressed

state. When a second free finger lands, the one-finger drag ends and the system starts a pinch instead.

Use `ZoomDelta()` for zoom that works across profiles: it returns mouse-wheel motion on desktop and the change in distance between two fingers on touch. `PanDelta()` reports movement of the midpoint between two fingers. `WheelMove()` remains a raw wheel query and has no touch equivalent.

The Knight `FlyThroughCamera`, `OrthogonalCamera`, `FollowUpCamera`, and `TopDownCamera` use `ZoomDelta()`. `FlyThroughCamera` also uses `PanDelta()`. This lets the same camera control respond to a mouse wheel or touch gesture without separate platform checks.

Keeping key-based code compatible

When an on-screen control is held, the keys associated with it behave as virtual key presses. A button presses the keys bound to its action, and a key pad presses the directional keys bound to its axis. Knight-derived classes can continue to call the protected `IsKeyDown()`, `IsKeyPressed()`, and `IsKeyReleased()` wrappers; those wrappers read physical and virtual keys through Input.

Code outside a Knight-derived class should use `KnightInput()`. The same applies to mouse wrappers such as IsMouseButtonDown(), GetMousePosition(), and `GetMouseDelta()`. Calls directly to raylib, including `::IsKeyDown()`, see physical input only. Raylib's built-in `UpdateCamera()` modes also read raylib directly, so they cannot be driven by Knight's virtual controls without replacing or adapting that camera input.

Testing touch controls on a development computer

Touch emulation displays the sample's virtual controls and lets the mouse act as one finger. Press F4 while running a desktop sample to switch it on; press F4 again to return to keyboard and mouse. The keyboard remains active while emulation is on.

With emulation enabled, click a button to press it, drag on a pad to steer, or drag elsewhere to move the pointer. Hold the mouse still for a long press. The mouse wheel

stands in for pinch zoom. Because a mouse supplies only one pointer, it cannot test two-finger panning or simultaneous pad-and-button use.

Touch emulation can also be selected at startup with `Config.EmulateTouch` = true, or with `KNIGHT_INPUT_PROFILE`=touch on desktop.

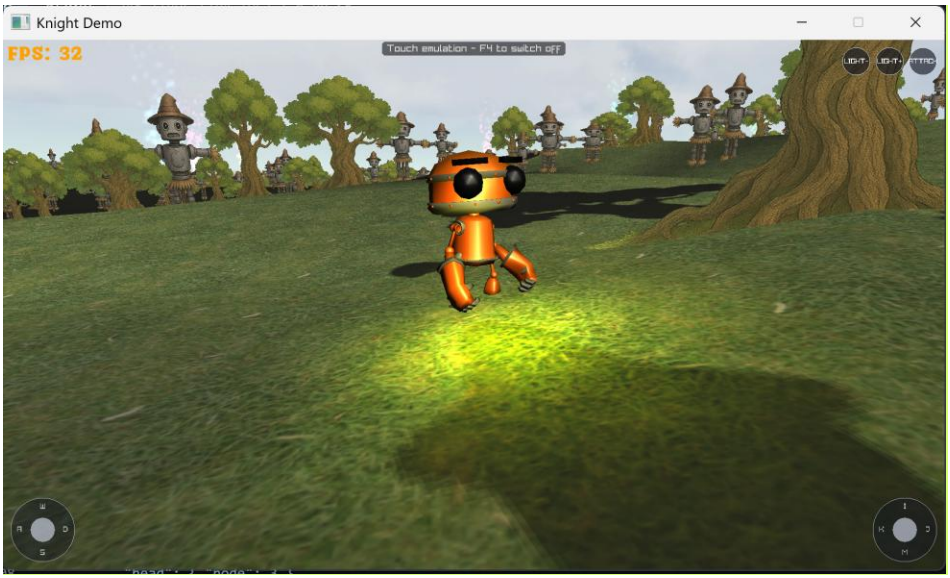


Figure 4 – Emulate on-screen GamePad and button controls on Desktop PC

The debug overlay helps diagnose control assignment and layout. Press F3 on or set `Config.ShowInputDebug` = true (the Android option). The panel shows the profile, action and axis values, pointers, zoom, and pan. Colored rings identify fingers assigned to the pointer, a control, or a gesture; display outlines show the safe area, control area, and fold hinge on the coming soon Android build.

How the input profile is selected

The `InputSystem` selects a default profile for the platform, then keeps the same actions, axes, and virtual controls if the profile changes. The profile controls whether virtual controls are shown and which pointer and zoom sources are used.

- Android starts with `Touch`. (Build for Android coming soon!)

- Desktop starts with `KeyboardMouse` unless `KNIGHT_INPUT_PROFILE`=touch is set.
- Web browsers with a coarse pointer default to Touch; browsers with a mouse default to KeyboardMouse.

A sample can also change profiles through `Input.SetProfile()`. Switching profiles releases controls that were held, which prevents a key or button from remaining stuck after the input source changes.

Conclusion

The `InputSystem` lets a game describe what the player is doing rather than which device produced the input. Named actions and axes give gameplay code a stable interface, while bindings connect that interface to keys, mouse input, touch pads, and buttons.

For simple samples, direct raylib input calls remain a clear choice. When a game needs remapping, touch controls, pointer gestures, or consistent behavior across platforms, the Knight input layer provides those features while keeping existing key-based code usable through its wrappers.

Challenging yourself

Add a top-to-bottom swipe gesture to a sample. Treat a mouse drag from the upper part of the screen to the lower part as the desktop version, and a single-finger drag as the touch version. Decide how far the pointer must travel before the system reports a swipe, so a very short movement or long press is not mistaken for the gesture. Then test the gesture with touch emulation (and soon you can test on a device with a real touch screen).

Happy coding!